

Aprendizaje de Máquina

U2 Análisis de Clústeres

Héctor Maravillo

Section 1

Center-based algorithms

m-Clustering

A **(hard) *m*-clustering** of a data set $X_{n \times d}$ is defined by a set of functions

$$u_j : X \rightarrow \{0, 1\}, \quad j = 1, \dots, m$$

where m is the number of clusters which is assumed to be known.

Let

- θ_j be the **parameterized representative** of the j -th cluster,
 $\Theta = (\theta_1^T, \dots, \theta_m^T)^T$
- U is an $n \times m$ matrix whose (i, j) elements equals $u_j(x_i)$
- $d(x_i, \theta_j)$ is a dissimilarity measure between x_i and θ_j

m -Clustering

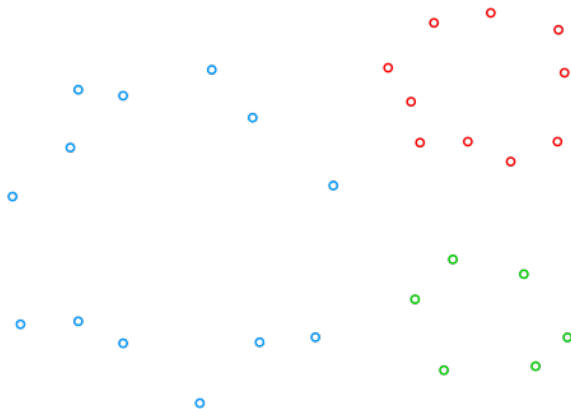


Figure: Data set $X_{n \times d}$

m -Clustering

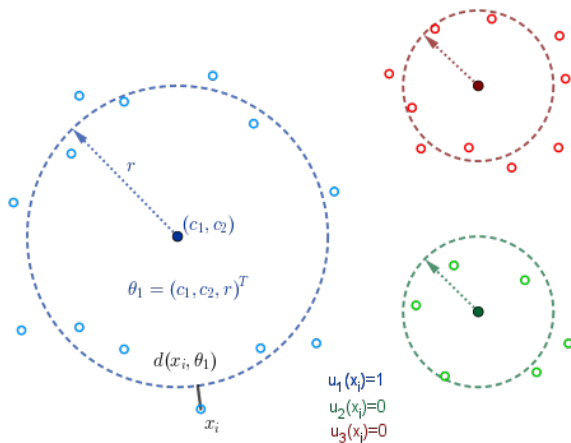


Figure: Parameterized representatives

Cost function

Most of the well-known clustering algorithms are those derived by **minimizing a cost function** of the form

$$J(\Theta, U) = \sum_{i=1}^n \sum_{j=1}^m u_{ij} d(x_i, \theta_j)$$

with respect to Θ and U , **subject to the constraints**

- Each data point is assigned to only one cluster:

$$\sum_{j=1}^m u_{ij} = 1, \quad i = 1, \dots, n$$

- No cluster is empty nor does it contain all the data points:

$$0 < \sum_{i=1}^n u_{ij} < n, \quad j = 1, 2, \dots, m$$

Cost function

Let us fix θ_j , $j = 1, \dots, m$. Since for each vector x_i only one u_{ij} is 1 and all the others are 0, it is straightforward to see that $J(\Theta, U)$ is **minimized** if we assign each x_i to its **closest cluster**, that is,

$$u_{ij} = \begin{cases} 1, & \text{if } d(x_i, \theta_j) = \min_{k=1, \dots, m} d(x_i, \theta_k) \\ 0, & \text{otherwise} \end{cases}, \quad i = 1, \dots, n$$

Section 2

k-Means

k-Means

The **Isodata** or **k-Means** or **c-Means** algorithm is one of the most popular and well-known clustering algorithms.

It can be viewed as a special case of a hard clustering algorithm when **point representatives (centroids)** are used and the **squared Euclidean “distance”** is adopted to measure the dissimilarity between the vectors x_i and the cluster representatives θ_j .

k-Means

For the k -Means the **cost function** becomes

$$J(\Theta, U) = \sum_{i=1}^n \sum_{j=1}^m u_{ij} \|x_i - \theta_j\|_2^2$$

The **isodata algorithm** recovers clusters that are as **compact** as possible (**converges to a minimum** of the cost function).

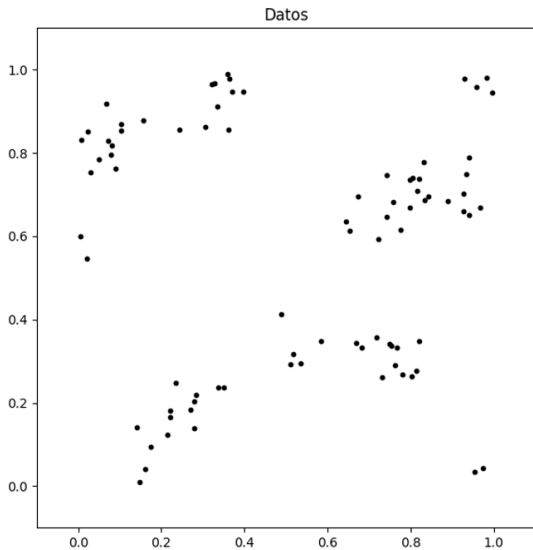
It must be emphasized however, that this convergence result is not valid for other distance including the Euclidean distance.

The k -Means Algorithm

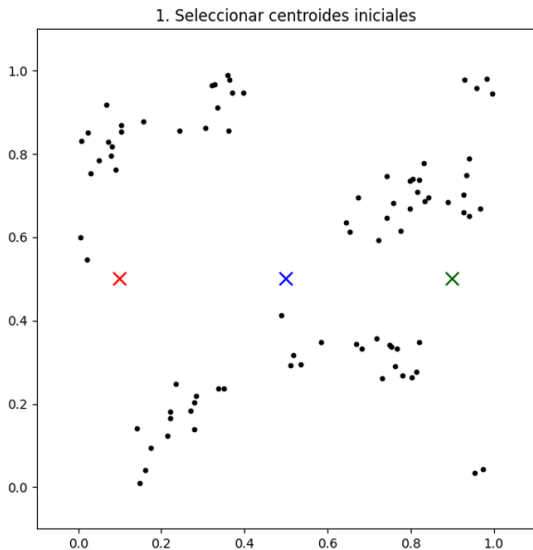
The k -Means Algorithm

- Choose **arbitrary initial** estimates (centroids) $\theta_j(0)$ for the θ_j 's, $j = 1, \dots, m$.
- Repeat
 - For $i = 1$ to n
 - Determine the **closest representative**, say θ_j for x_i .
 - Set $b(i) = j$
 - For $j = 1$ to m
 - **Parameter updating:** Determine θ_j as the mean of the vectors $\{x_i \in X \mid b(i) = j\}$
- **Until no change** in θ_j 's occurs between two successive iterations.

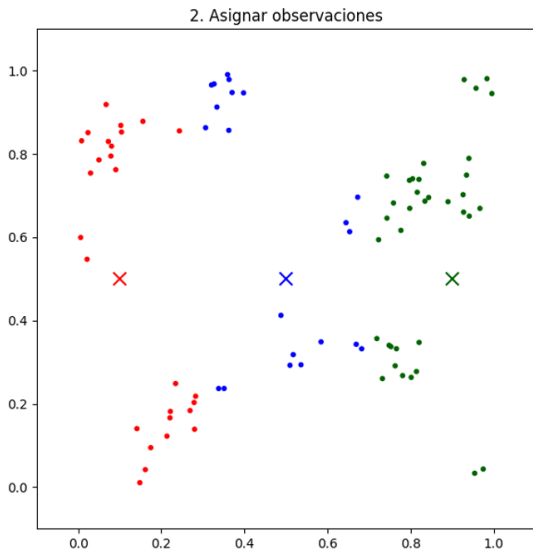
Example



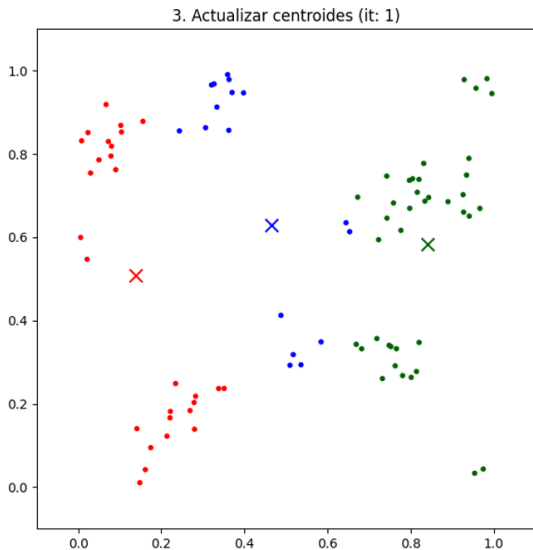
Example



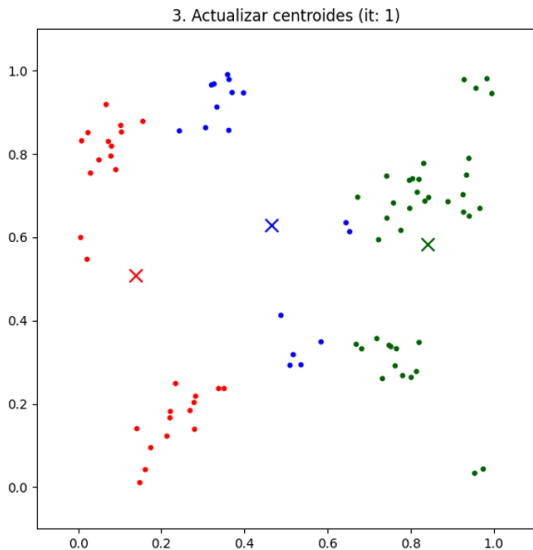
Example



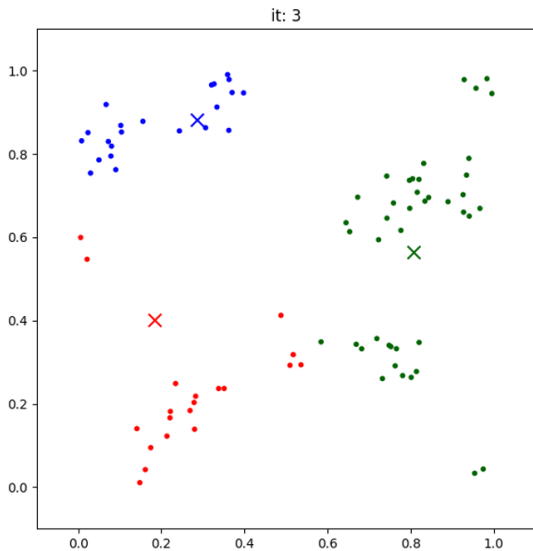
Example



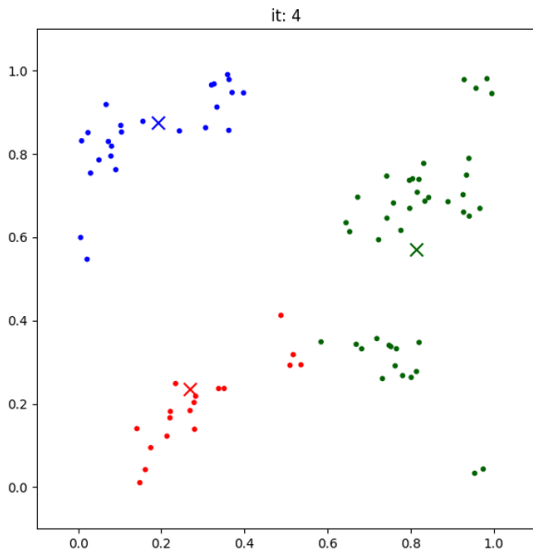
Example



Example



Example



Advantages

- **Computational simplicity.** Its time complexity is $O(nmq)$ where q is the number of iterations required for convergence (at local minimum). Because, in practice m and q are significantly less than n , k -Means becomes eligible for processing **large data sets**.
- **Conceptual simplicity.**

Drawbacks

- **Different initial partitions** may lead k -Means to produce **different final clustering**, each corresponding to a different **local minimum** of $J(\Theta, U)$.
- The **number of clusters** m in the data set, X is required as an **input parameter**. A poor estimate of M will prevent the algorithm to unravel the underlying clustering structure in X .
- k -Means is **sensitive** to **outliers** and **noise**. The outliers, being points in X are necessarily assigned to one of the clusters. Thus, they influence the respective means and, as a consequence, the final clustering.
- K -means is generally applicable to data sets with continuous valued feature vectors, and in principle it **is not suitable** for data with **nominal (categorical)** coordinates.

Section 3

k -Medoids Algorithms

k -Medoids Algorithms

- In the k -**Means algorithm**, each cluster is represented by the mean of its vectors (**centroid**).
- In the k -**Medoids** methods, each cluster is represented by a vector selected among the elements of X (the closest to all the others), and we will refer to it as **the medoid**.

k-Medoids Algorithms

Apart from its **medoid**, each cluster contains all vectors in X that

- Are not used as medoids in other clusters
- **Lie closer** to its medoid than to the medoids representing the other clusters.

Let Θ be the **set of medoids** for all clusters.

We will denote by I_{Θ} the set of **indices** of the points in X that constitute Θ , and by $I_{X-\Theta}$ the set of **indices** of the points that are not medoids.

Thus, if for example $\Theta = \{x_1, x_5, x_{13}\}$ is the set of medoids for a three-cluster case then $I_{\Theta} = \{1, 5, 13\}$.

Quality function of k -Medoids

The **quality** of the clustering associated with a given set of medoids is assessed through the **cost function**

$$J(\Theta, U) = \sum_{i \in I_{X-\Theta}} \sum_{j \in I_{\Theta}} u_{ij} d(x_i, x_j)$$

and

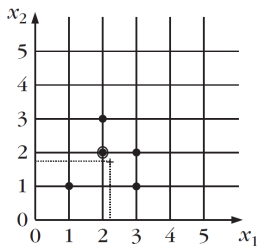
$$u_{ij} = \begin{cases} 1, & \text{if } d(x_i, x_j) = \min_{q \in I_{\Theta}} d(x_i, x_q) \\ 0, & \text{otherwise} \end{cases}$$

Thus, obtaining the set of medoids Θ that best represents the data set, X , is equivalent **to minimizing** $J(\Theta, U)$.

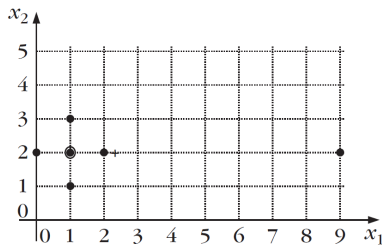
Advantages

Representing clusters using medoids has two **advantages** over the k -Means algorithm:

- It can be used with data sets originating from either **continuous** or **discrete domains**, whereas k -Means is suited only for the case of continuous domains because in mean of a subset of the data vectors is not necessarily a point lying in a discrete domain.
- k -Medoids algorithms tend to be **less sensitive to outliers** compared to the k -Means algorithm.



(a)



(b)

Disadvantages

- The mean of a cluster has a clear geometrical and statistical meaning, which is not necessarily the case with the medoids.
- The algorithms for the estimation of the best set of medoids are **computationally more demanding** compared to the k -Means algorithm.

Section 4

The PAM Algorithm

The PAM Algorithms

To determine the set Θ of the m medoids that best represent the data set, **Partitioning Around Medoids (PAM) algorithm** uses a function optimization procedure that minimizes $J(\Theta, U)$, subject to the constraint that the representatives of the clusters are themselves elements of X .

Definitions

Two sets of medoids Θ and Θ' , each consisting of m elements, are called **neighbors** if they share $m - 1$ elements.

Clearly, the number of neighbors a set $\Theta \subset X$ with m elements can have is $m(N - m)$.

Also, let Θ_{ij} denote the neighbor of Θ that results if x_j , $j \in I_{X-\Theta}$ replaces x_i , $i \in I_{\Theta}$. The term U_{ij} refers the new assignation of instance to the medoids set Θ_{ij} .

Finally, let

$$\Delta J_{ij} = J(\Theta_{ij}, U_{ij}) - J(\Theta, U)$$

Definitions

For example, if in a data set $X = \{x_1, x_2, x_3, x_4, x_5\}$, the set of medoids is

$$\Theta = \{x_1, x_3\}$$

Then, the neighbors are

$$\Theta_{12} = \{x_2, x_3\}$$

$$\Theta_{32} = \{x_1, x_2\}$$

$$\Theta_{14} = \{x_4, x_3\}$$

$$\Theta_{34} = \{x_1, x_4\}$$

$$\Theta_{15} = \{x_5, x_3\}$$

$$\Theta_{35} = \{x_1, x_5\}$$

PAM Algorithm

- PAM starts with a set Θ of m medoids, which are **randomly selected out** of X .
- Then, among all $m(N - m)$ neighbors,

$$\Theta_{ij}, \quad i \in I_{\Theta}, j \in I_{X-\Theta}$$

of the set Θ , we select

$$\Theta_{qr}, \quad q \in I_{\Theta}, r \in I_{X-\Theta}$$

with

$$\Delta J_{qr} = \min_{ij} \Delta J_{ij}$$

PAM Algorithm

- If ΔJ_{qr} is negative, then Θ is replaced by Θ_{qr} and the same procedure is repeated.
- Otherwise, if $\Delta J_{qr} \geq 0$ the algorithm has reached a local minimum and terminates.
- Once the set Θ that best represents the data has been determined, each $x \in X - \Theta$ is assigned to the cluster represented by the closest to its medoid.

Complexity

Experimental results show that PAM works satisfactorily for relatively **small data sets**.

However, it becomes inefficient for **large data sets** because its time complexity per iteration increases **quadratically** with respect to N . This is easily verified because at each iteration the term ΔJ_{ij} for $(N - m)$ pairs of vectors has to be calculated.

The total complexity of the algorithm per iteration amounts to $O(m(N - m)^2)$.

Section 5

CLARA and CLARANS

CLARA and CLARANS: k -Medoids Algorithm for Large Data Sets

These algorithms are versions of the PAM algorithm, and they have been developed to handle with the **high computational demands** imposed by **large data sets**.

Both algorithms exploit the idea of **randomized sampling** but each in a different way

CLARA Algorithm

The idea underlying **Clustering LARge Applications (CLARA)** is to **draw randomly** a sample X' of size N' from the entire data set, X , and to determine the set Θ' of the medoids that best represents X' using the **PAM algorithm**.

The rationale behind this algorithm is based on the assumption that if the sample X' is drawn in a way that is **representative of the statistical distribution** of the data points in X the set will be a satisfactory approximation of the set of the medoids that would result if the PAM algorithm was run on the entire data set.

CLARA Algorithm

To obtain better results, CLARA **runs PAM** a number of sample subsets of X , denoted by $X'_1, \dots, X'_s$. Each run returns a set of medoids denoted by $\Theta'_1, \dots, \Theta'_s$.

Then, the quality of the clustering associated with each of them is calculated, where the entire data set, X , is taken into account.

Experimental studies suggest that $s = 5$ and $N' = 40 + 2m$ lead to satisfactory results.

CLARANS Algorithm

According to **Clustering Large Applications based on RANdomized Search (CLARANS)**, PAM is applied on the entire data set, X , but with a slight modification.

At each iteration, not all neighbors of the current set of medoids are considered. Instead, only a **randomly selected fraction** $q < m(N - m)$ of them is utilized.

CLARANS Algorithm

The **selected neighbors** are considered in a **sequential manner** and if the currently considered neighbor Θ_{ij} of Θ is better than Θ (in terms of J) then Θ is replaced by Θ_{ij} and the procedure is repeated.

When none of the q selected neighbors of Θ is better than Θ , in terms of J , then is considered to be **local minimum**.

Then CLARANS **starts from another** arbitrarily chosen Θ , and the same procedure is followed in order to obtain a different *local minimum*.

This is repeated for a predetermined number of times, s , and the algorithm outputs the best among the s **local minimum**.

In the sequel, based on this set of medoids each point $x \in X - \Theta$ assigned to the cluster whose representative is closest to x .

Section 6

References

References

- Theodoridis, S. & Koutroumbas, K. *Pattern Recognition*, 4th ed., Academic Press, 2009